Sometimes you'll want to execute some C#, but you don't need to output the values. We used this technique when we were setting values in `ViewData`:

```
@{
    ViewData["Title"] = "Home Page";
}
```

This example demonstrates a *Razor code block*, which is normal C# code, identified by the `@{}` structure. Nothing is written to the HTML output here; it's all compiled as though you'd written it in any other normal C# file.

TIP When you execute code within code blocks, it must be valid C#, so you need to add semicolons. Conversely, when you're writing values directly to the response using Razor expressions, you don't need them. If your output HTML breaks unexpectedly, keep an eye out for missing or rogue extra semicolons.

Razor expressions are one of the most common ways of writing data from your `PageModel` to the HTML output. You'll see the other approach, using Tag Helpers, in the next chapter. Razor's capabilities extend far further than this, however, as you'll see in section 17.3.2, where you'll learn how to include traditional C# structures in your templates.

17.3.2 Adding loops and conditionals to Razor templates

One of the biggest advantages of using Razor templates over static HTML is the ability to generate the output dynamically. Being able to write values from your `PageModel` to the HTML using Razor expressions is a key part of that, but another common use is loops and conditionals. With these, you can hide sections of the UI, or produce HTML for every item in a list, for example.